SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I _Lavanya.M(192321051)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

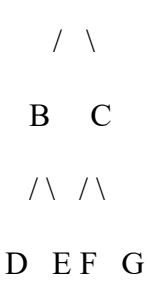
Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



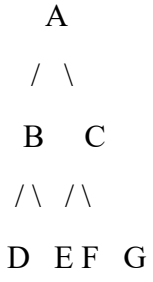
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
   1 2 3
   4 5 6
   7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Question 1 – Breadth-First Search (BFS)

Given graph: A is the root; B and C are its children; D and E are children of B; F and G are children of C. Assume children are visited from left to right.
BFS visits nodes level by level. Start with queue [A]. At each iteration, remove the front node and append its unvisited children to the rear.

Iteration	Queue before visiting node	Visited node; queue after expansion
1	[A]	Visit A; queue becomes [B, C]
2	[B, C]	Visit B; queue becomes [C, D, E]
3	[C, D, E]	Visit C; queue becomes [D, E, F, G]
4	[D, E, F, G]	Visit D; queue becomes [E, F, G]
5	[E, F, G]	Visit E; queue becomes [F, G]
6	[F, G]	Visit F; queue becomes [G]
7	[G]	Visit G; queue becomes []

Answers

1. BFS traversal order: A → B → C → D → E → F → G.
2. Queue contents after each iteration: [B, C] → [C, D, E] → [D, E, F, G] → [E, F, G] → [F, G] → [G] → [].

Question 2 – Depth-First Search (DFS)

Assume left-to-right traversal. DFS follows one branch as deeply as possible before backtracking.
Initial state: stack contains [A] (A is the top/next node to visit, depending on stack notation).

Step	Node visited	Action / stack idea
1	A	Visit A; explore left child B first
2	B	Visit B; explore left child D
3	D	D is a leaf; backtrack to B
4	E	Visit E; E is a leaf; backtrack to A
5	C	Explore A's right child C
6	F	Visit F; F is a leaf; backtrack to C
7	G	Visit G; G is a leaf

DFS traversal order: A → B → D → E → C → F → G.

Question 3 – 8-Puzzle Using BFS

Initial state:
1 3 6\n5 0 2\n4 7 8
Goal state:
1 2 3\n4 5 6\n7 8 0
BFS explores states in increasing order of depth. The blank (0) moves one square at a time. The first five queue expansions are shown below. For compactness, each board is written row-wise with "/" between rows; for example, 136/502/478 means rows 1 3 6; 5 0 2; 4 7 8. Move generation order is Up, Down, Left, Right.

Iteration	Current state expanded	Queue before expansion (front first)	Queue after expansion (front first)
1	136/502/478	136/502/478	106/532/478, 136/572/408, 136/052/478, 136/520/478
2	106/532/478	106/532/478, 136/572/408, 136/052/478, 136/520/478	136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478
3	136/572/408	136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478	136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480
4	136/052/478	136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480	136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078
5	136/520/478	136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078	016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078, 130/526/478, 136/528/470

Answers to the 8-puzzle questions

1. The first five expanded nodes/states are: 136/502/478, 106/532/478, 136/572/408, 136/052/478, and 136/520/478. BFS continues beyond these five iterations before reaching the goal.
2. With the stated Up, Down, Left, Right successor order and counting the initial state as generated, 500 distinct states are generated/discovered by the time the goal state is dequeued. The goal is reached after 8 moves. Counts can vary if a different successor-generation order or counting convention is used.
3. One shortest solution path (blank moves) is: Right → Up → Left → Down → Left → Down → Right → Right.

Step	Board state
Initial	1 3 6\n5 0 2\n4 7 8
1. Right	1 3 6\n5 2 0\n4 7 8
2. Up	1 3 0\n5 2 6\n4 7 8
3. Left	1 0 3\n5 2 6\n4 7 8
4. Down	1 2 3\n5 0 6\n4 7 8
5. Left	1 2 3\n0 5 6\n4 7 8
6. Down	1 2 3\n4 5 6\n0 7 8
7. Right	1 2 3\n4 5 6\n7 0 8
8. Right (Goal)	1 2 3\n4 5 6\n7 8 0

4. BFS finds a shortest solution in an unweighted 8-puzzle because every legal move has equal cost. BFS expands all states at depth 0, then depth 1, depth 2, and so on. Therefore, the first time it reaches the goal, it has used the minimum possible number of moves.

Important note

The evaluation sheet provides only five rows for the 8-puzzle trace, but the goal is not reached within the first five expansions. The table above fills those five rows and the shortest path is provided separately. A complete queue-by-queue trace all the way to the goal would require many more rows.